



Session State & Resumption

การจัดการสถานะและการกลับมาทำงานต่อของ agent

Domain 1 · Task 1.7 — Claude Certified Architect

Session Management คืออะไร?

- คือกลไกที่กำหนดว่า agent จะรักษา **ความต่อเนื่อง (continuity)** ของงานข้ามหลาย session อย่างไร
- สำคัญกับงานยาว ๆ ที่กินเวลาหลายวัน



debugging



code review



research หลายวัน

ประเด็นหลักของ Task: เมื่อจะ**กลับมาทำงานต่อ** ควรเลือกวิธีไหน และอะไรคือ**กั๊ก**ที่ทำให้ agent แะนำผิด

สามทางเลือกในการจัดการ Session

- 1 `--resume <session-name>` — กู้คืน**ประวัติสนทนาทั้งหมด** รวม tool results, การวิเคราะห์ และ reasoning เดิม
- 2 `fork_session` — แยกเป็น **branch อิสระ**จาก baseline ร่วมกัน · การเปลี่ยนใน branch หนึ่งไม่กระทบ branch อื่น
- 3 **Fresh start + summary injection** — เริ่ม session ใหม่ จัดสรุปที่**คัดกรองแล้ว**เข้า context · ไม่มี tool result เก้าค้างมา

เลือกทางไหนเมื่อไหร่

`--resume`

✓ ใช้เมื่อ context เดิมยังถูกต้องและไฟล์ยังไม่เปลี่ยนมาก · ✗ เลี่ยงเมื่อ ไฟล์ถูกแก้ไขไปแล้ว (เกิด stale context)

`fork_session`

✓ ใช้เมื่อ ต้องสำรวจหลายแนวทางจากจุดเริ่มร่วมกัน · ✗ เลี่ยงเมื่อ แค่ทำงานเดิมต่อในเส้นเดียว

Fresh start + summary

✓ ใช้เมื่อ tool results เดิมล้าสมัยหรือ context เสื่อมคุณภาพ · ✗ เลี่ยงเมื่อ context เดิมยังใช้ได้และยังต้องการประวัติเดิม

The Stale Context Problem

หัวใจของ Task นี้

ปัญหาเกิดเมื่อ agent กลับมาทำงานต่อ**หลังจากได้ถูกแก้ไขไปแล้ว** แต่ยัง reason จาก tool results ที่ถูก **cache ไว้**

ซึ่งไม่สะท้อนสถานะปัจจุบันของไฟล์

 **ทำไม** `--resume` **ถึงพัง**

เก็บ**ประวัติสนทนาทั้งหมด** รวมเนื้อไฟล์เก่าในรูป tool result

เมื่อไฟล์เปลี่ยน → ข้อมูลเก่าที่ล้าสมัย**ยังคงอยู่บน**กับข้อมูลใหม่



อาการของ Stale Context

★ ออกสอบบ่อย

อาการ

- ให้คำแนะนำที่**ขัดแย้งกันเอง**เกี่ยวกับไฟล์ที่ถูกแก้
- แนะนำให้ทำการเปลี่ยนแปลงที่**ได้ทำไปแล้ว**
- อ้างอิงโค้ดที่**ไม่มีอยู่ในไฟล์อีกต่อไป**

สาเหตุ

`--resume` รักษาประวัติทั้งหมดไว้ รวมเนื้อไฟล์เก่าที่เป็น tool result

เมื่อไฟล์เปลี่ยน ข้อมูลล่าสุดยังอยู่**เคียงข้าง**ข้อมูลใหม่

วิธีแก้ที่ผิด vs ที่ถูก

✗ แบบ naive (ไม่พอ)

แค่ `--resume` แล้วสั่งให้ agent อ่านไฟล์ที่แก้ไขใหม่ซ้ำ
ผลเก่าที่ค้างในประวัติ**ยังมีอิทธิพล**ต่อการตัดสินใจ

✓ วิธีแก้ที่ถูกต้อง

เริ่ม **fresh session** พร้อม structured summary
และระบุชัดว่า**ไฟล์ไหนเปลี่ยน**
→ ให้ agent ทำ **targeted re-analysis** เฉพาะไฟล์เหล่านั้น

Targeted Re-Analysis vs Full Re-Exploration

หลักวิธีวิจัย: เมื่อไฟล์เปลี่ยนไปแค่ส่วนน้อย อย่ารี้อวิเคราะห์ใหม่ทั้งหมด

1 เริ่ม **fresh session**

2 จัด structured summary: finding ที่ยังไม่เปลี่ยน + รายการไฟล์ที่เปลี่ยน

3 agent วิเคราะห์เฉพาะไฟล์ที่เปลี่ยน

4 รวมผลใหม่เข้ากับ finding เดิมที่ยังใช้ได้

เร็วกว่า full re-exploration และเชื่อถือได้กว่าการ resume กับ stale context

Decision Matrix

ทำงานต่อ ไม่มีไฟล์เปลี่ยน

`--resume`

เทียบหลายแนวทาง refactoring

`fork_session`

resume หลังแก้ 3 จาก 50 ไฟล์

`fresh + summary`

session ยาว ประสิทธิภาพ

`fresh + summary`

เทียบกลยุทธ์ testing vs documentation

`fork_session`

resume หลัง update dependency

`fresh + summary`



Case Study: คำแนะนำที่ขัดแย้งกันเอง

อาการ

Dev วิเคราะห์ codebase **50 ไฟล์** สองวัน เจอปัญหา authentication สามจุด → แก้ทั้งสามไฟล์ตอนกลางคืน
พอ `-resume` วันรุ่งขึ้น Claude กลับแนะนำให้แก้ปัญหาที่แก้ไปแล้ว และตอบขัดแย้ง อ้างถึงโค้ดเก่าและใหม่

✅ วิธีแก้

เริ่ม fresh session พร้อม summary:

"การวิเคราะห์เดิมเจอปัญหา *auth* สามจุดใน `auth.ts` , `session.ts` , `middleware.ts` — ทั้งสามแก้แล้ว ช่วยวิเคราะห์สามไฟล์นี้ซ้ำเพื่อยืนยันการแก้และตรวจปัญหาใหม่ที่จะเกิดขึ้น"



Exam Traps ที่ต้องระวัง

1. Full re-exploration trap

เสนอรีวิเคราะห์ทั้ง 50 ไฟล์ทั้งที่เปลี่ยนแค่ 3 → ที่ถูกคือ **targeted re-analysis** + summary

2. Resume after modifications trap

แนะนำ `--resume` หลังไฟล์เปลี่ยน → stale results ค้างทำให้ขัดแย้ง · **fresh + summary** เหนือกว่า

3. Confusing fork with resume trap

`fork_session` = branch อิสระต่างแนวทาง · `--resume` = ทำงานเดิมต่อในสันทนาเดียว — **คนละวัตถุประสงค์**

4. Using fork for stale context trap

`fork_session` สืบทอด stale tool results มาด้วย → ที่แก้ stale จริงคือ **fresh start + summary injection**



Practice Scenario

ลองคิดก่อนดูเฉลย

Dev ทำ `--resume` session ของ Claude Code หลังแก้ไข 3 ไฟล์ใน codebase 50 ไฟล์ · agent ให้คำแนะนำขัดแย้งกันเอง แนะนำสิ่งที่ทำไปแล้ว และอ้างโค้ดที่ไม่มีอยู่จริง

แนวทางที่เหมาะสมที่สุดคืออะไร?

A. `--resume` แล้วสั่งให้ agent อ่าน 3 ไฟล์ที่แก้ไขใหม่ซ้ำ

B. เริ่ม session ใหม่ทั้งหมด แล้ววิเคราะห์ทั้ง 50 ไฟล์ใหม่

C. เริ่ม fresh session จัด summary ระบุ 3 ไฟล์ที่เปลี่ยน เพื่อทำ targeted re-analysis

D. ใช้ `fork_session` แยก branch รวมการเปลี่ยนแปลงไฟล์แบบอิสระ

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option C

เริ่ม fresh session พร้อม structured summary และระบุ 3 ไฟล์ที่เปลี่ยน → agent ทำ **targeted re-analysis** เฉพาะไฟล์เหล่านั้น แล้วรวมกับ finding เดิมที่ยังใช้ได้
กำจัด stale context โดย**ไม่ต้องรีโหลดทั้งหมด**

A ผิด — ยังคงค้าง stale results เดิมในประวัติ การอ่านซ้ำไม่ลบข้อมูลเก่าที่มีอิทธิพล

B ผิด — ถูกทางแต่แพงและช้าเกิน รื้อทั้ง 50 ไฟล์ทั้งที่เปลี่ยนแค่ 3

D ผิด — `fork_session` สืบทอด stale tool results จาก session เดิม ไม่แก้ต้นเหตุ



Build Exercise

แบบฝึก 45 นาที — ฝึกจับ stale context ด้วยมือ

1 สร้าง named session วิเคราะห์ codebase 10 ไฟล์

2 บันทึก findings เป็น structured summary

3 แก้ 3 ไฟล์เพื่อ fix ปัญหาที่เจอ

4 ลอง `--resume` แล้วสังเกตอาการ stale

5 fresh session + จัด summary → targeted re-analysis

6 เทียบคุณภาพคำแนะนำ resume vs fresh

เห็นชัดว่าทำไม **summary injection** เหนือกว่า · Domain 1 · Task 1.7



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

Dev resume session ของ Claude Code หลังแก้ไข **3 ไฟล์** ใน codebase 50 ไฟล์ · agent ให้คำแนะนำขัดแย้งกันเอง แนะนำการเปลี่ยนแปลงที่ทำไปแล้ว และอ้างถึงโค้ดที่ไม่มีอยู่จริง · แนวทางที่เหมาะสมที่สุดคืออะไร?

- A. เริ่ม session ใหม่ทั้งหมด แล้ววิเคราะห์ทั้ง codebase 50 ไฟล์ใหม่จากศูนย์
- B. Resume session แล้วสั่งให้ agent อ่านซ้ำเฉพาะ 3 ไฟล์ที่แก้ เพื่ออัปเดตความเข้าใจ
- C. เริ่ม fresh session ดึง summary ของ finding เดิม พร้อมบอก 3 ไฟล์ที่เปลี่ยนเพื่อทำ **targeted re-analysis**
- D. ใช้ `fork_session` แยก branch ใหม่ที่รวมการเปลี่ยนแปลงไฟล์แบบอิสระ

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option C

Fresh session + summary injection กำจัด stale tool results ทั้งทั้งหมด · summary เก็บความรู้จาก session เดิมไว้ · การระบุ 3 ไฟล์ที่เปลี่ยนช่วยให้ทำ targeted re-analysis ได้โดยไม่ต้องรื้อทั้ง codebase

ทำไมข้ออื่นผิด

A — รื้อทั้ง 50 ไฟล์เปลืองเปล่า ทั้งที่เปลี่ยนแค่ 3 · การวิเคราะห์อีก 47 ไฟล์ยังใช้ได้ · targeted re-analysis คุ่มกว่า

B — resume ยังเก็บ stale tool results ไว้ในประวัติ · แม้อ่าน 3 ไฟล์ซ้ำ เนื้อไฟล์เก่ายังค้างใน context และมีอิทธิพลต่อการตัดสินใจที่เกี่ยวข้อง

D — `fork_session` แยก branch จาก session เดิมที่ยังมี stale results · fork สืบทอด stale context เดิม · fork ไว้สำหรับ divergent exploration ไม่ใช่แก้ stale data



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

หลังวิเคราะห์ codebase เสร็จ Dev ต้องการเปรียบเทียบ refactoring สองแนวทาง: แนวหนึ่งเน้น performance อีกแนวเน้น readability · แนวทางจัดการ session แบบใดเหมาะสมที่สุด?

- A. เริ่ม session ใหม่ทั้งหมดสองอัน อันละแนวทาง
- B. ใช้ `--resume` ทำงานต่อใน session เดิม แล้วลองทั้งสองแนวทางตามลำดับ
- C. ใช้ `fork_session` แยกสอง branch อีกระยะจาก analysis baseline ที่ใช้ร่วมกัน
- D. เริ่ม fresh session พร้อม summary injection แยกแต่ละแนวทาง

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option C

`fork_session` สร้าง branch อีกระยะจาก analysis baseline ที่ใช้ร่วมกัน · ทั้งสอง branch ต่อยอดจากความเข้าใจ codebase ตั้งต้นเดียวกัน แต่แยกสำรวจคนละทิศทางอย่างอิสระ — เป็น use case ตรงตัวของ fork

ทำไมข้ออื่นผิด

A — เริ่ม session ใหม่ทั้งหมดทำให้เสีย analysis ตั้งต้น · ทั้งสองแนวทางควรได้ประโยชน์จาก baseline ความเข้าใจร่วมกัน

B — ลองสองแนวทางใน session เดียวทำให้การสำรวจปนเปื้อน · แนวทางที่สองถูกอิทธิพลจาก analysis และการตัดสินใจของแนวทางแรก

D — summary injection เก็บความรู้ได้บางส่วน แต่ `fork_session` เก็บ analysis context ครบถ้วนกว่า และเป็นกลไกที่ออกแบบมาเพื่อ divergent exploration โดยเฉพาะ



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

Dev วิเคราะห์ codebase เสร็จเมื่อวาน · ข้ามคืนไม่มีไฟล์ไหนเปลี่ยน · วันนี้อยากทำงานต่อจากจุดที่หยุด · แนวทางใดดีที่สุด?

A. เริ่ม fresh session พร้อม summary injection

B. ใช้ `--resume <session-name>` ทำงานต่อใน named session เดิม

C. ใช้ `fork_session` แยก branch จาก analysis ของเมื่อวาน

D. เริ่ม session ใหม่ทั้งหมด แล้ววิเคราะห์ทุกอย่างใหม่

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

ไม่มีไฟล์เปลี่ยน context เดิมจึงยังใช้ได้ · `--resume` คืนประวัติสนทนาทั้งหมด ให้ Dev ทำงานต่อจากจุดที่หยุดได้เป๊ะ — เป็นสถานการณ์ในอุดมคติของ resume

ทำไมข้ออื่นผิด

- A — summary injection สูญรายละเอียดจากประวัติสนทนาเดิม · เมื่อ context ยังใช้ได้ resume เก็บข้อมูลได้มากกว่าที่ summary จะจับไหว
- C — fork ไว้สำหรับ divergent exploration ไม่ใช่การทำงานต่อธรรมดา · Dev ต้องการสานงานเดิม ไม่ได้สำรวจทางเลือกใหม่
- D — วิเคราะห์ทุกอย่างใหม่เปลืองเปล่า ทั้งที่ analysis เดิมยังใช้ได้ · resume คุ่มกว่า



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

ทำไมการ resume session หลังแก้ไขไฟล์จึงมีปัญหามากกว่าการสั่งให้ agent อ่านไฟล์ที่เปลี่ยนซ้ำเฉย ๆ?

A. agent ไม่สามารถอ่านไฟล์ซ้ำได้ระหว่าง resumed session

B. stale tool results จาก session ก่อนยังค้างในประวัติศาสตร์ และมียกเว้นต่อการตัดสินใจที่เกี่ยวข้อง แม้จะอ่านไฟล์ที่เปลี่ยนซ้ำแล้ว

C. resumed session มี context window ที่เล็กลง จนรับเนื้อไฟล์ใหม่ไม่ไหว

D. agent จะปฏิเสธไม่ยอมวิเคราะห์ถ้าตรวจพบว่าการแก้ไขไฟล์

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

Resume เก็บประวัติสนทนาทั้งหมดไว้ รวมถึง tool results เท่าที่แสดงเนื้อไฟล์ก่อนแก้ · แม้อ่านไฟล์ที่เปลี่ยนซ้ำ เนื้อเก่า
ยังค้างใน context · agent อาจอ้าง stale data โดยเฉพาะการตัดสินใจข้างเคียงที่ไม่เกี่ยวกับไฟล์ที่แก้โดยตรง

ทำไมข้ออื่นผิด

- A — agent อ่านไฟล์ซ้ำได้ระหว่าง resumed session · ปัญหาไม่ใช่ความสามารถ แต่คือ context ปนเปื้อนจาก stale data
- C — resumed session ไม่ได้มี context window เล็กลง · ปัญหาคือ stale data ในประวัติเดิม ไม่ใช่ความจุ context
- D — agent ไม่ได้ตรวจจับหรือปฏิเสธการวิเคราะห์เพราะการแก้ไฟล์ · มันแค่ให้เหตุผลจากข้อมูลที่อยู่ใน context รวมถึง stale results

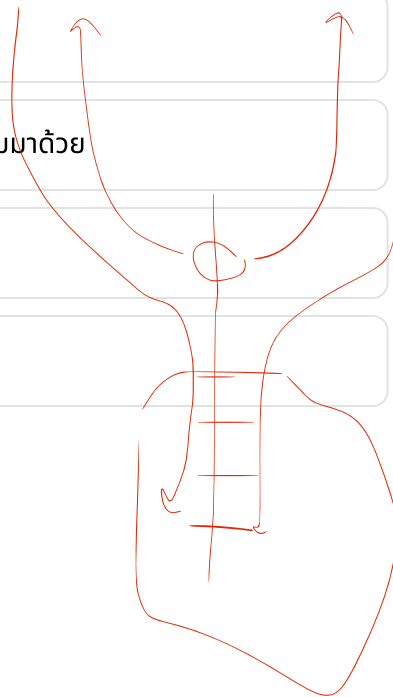


Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

Dev ใช้ `fork_session` เพื่อจัดการ stale context หลังแก้ไขไฟล์ · ทำไมแนวทางนี้จึงมีข้อบกพร่อง?

- A. `fork_session` ใช้ไม่ได้หลังจากมีการแก้ไขไฟล์
- B. `fork_session` แยก branch จาก session เดิมที่ยังมี stale tool results — fork จึงสืบทอด stale context เดิมมาด้วย
- C. `fork_session` ลบ session เดิมทิ้ง ทำให้เสีย analysis ก่อนหน้าทั้งหมด
- D. `fork_session` ใช้ได้แค่การเปรียบเทียบ testing strategies ไม่ใช่การใช้งานทั่วไป

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป





เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

`fork_session` แยก branch จาก state ปัจจุบันของ session รวมถึง stale tool results ทั้งหมด · fork สืบทอด context ปนเปื้อนเดิมที่ก่อคำแนะนำขัดแย้ง · fork ไม่ได้สร้าง clean slate — มันคัดลอก state ปัจจุบัน · สำหรับ stale context ที่ถูกคือ fresh start + summary injection

ทำไมข้ออื่นผิด

- A — `fork_session` ใช้ได้ไม่ว่าจะแก้ไฟล์หรือไม่ · ปัญหาไม่ใช่ความพร้อมใช้งาน แต่คือการสืบทอด stale data
- C — `fork_session` ไม่ได้ลบ session เดิม · มันแยก branch อีกระยะโดยยังเก็บ original ไว้
- D — `fork_session` ใช้กับ divergent exploration ได้ก็ดี · แต่มันผิดสำหรับจัดการ stale context โดยเฉพาะ เพราะสืบทอด stale data